

Dockerized Marketing PHP Laravel Service

Course Name: DevOps

Institution Name: Medicaps University - Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	CHARU RATHOD	EN23CS3T1007
2.	RUDRAKSH SHARMA	EN22CS301832
3.	PRATHAM DESHMUKH	EN22CS301741
4.	PIYUSH PATIDAR	EN22CS301705
5.	ROHAN JAIN	EN22CS301817
6.	PRIYANSH MEWADA	EN22CS301763

*Group Name:*G03D7

*Project Number:*DO-26

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Patel

Academic Year: 2025-2026

Table of Contents

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

Results & Output

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

Conclusion

Future Scope & Enhancements

Problem Statement & Objectives

1. Problem Statement

In modern software development, deploying web applications efficiently and consistently across multiple environments remains a major challenge. Traditional deployment approaches often require manual configuration of servers, installation of dependencies, and environment setup, which leads to inconsistencies between development, testing, and production environments. These inconsistencies often result in unexpected bugs, system failures, and delays in deployment.

Another significant issue is scalability. As user demand increases, systems built using traditional architectures struggle to handle high loads effectively. Additionally, dependency conflicts, lack of automation, and time-consuming setup processes further reduce productivity and increase operational costs.

To overcome these challenges, there is a need for a robust, scalable, and automated solution that ensures consistency across environments, simplifies deployment, and enhances system performance. This project addresses these problems by implementing a containerized architecture using Docker, along with CI/CD pipelines and Kubernetes orchestration.

The Dockerized Marketing PHP Laravel Service is designed to eliminate environment-related issues, automate deployment processes, and provide a scalable infrastructure that can handle increasing workloads efficiently. By leveraging DevOps practices, this project ensures faster delivery, improved reliability, and better resource utilization.

2. Project Objectives

The main objective of this project is to build a scalable and automated deployment system using modern DevOps tools.

Objectives:

- Containerize Laravel application using Docker
- Implement CI/CD pipelines
- Deploy application on Kubernetes
- Improve performance using Redis
- Ensure secure and reliable system

3. Scope of the Project

The project focuses on developing a Dockerized Laravel-based marketing service with a scalable and consistent containerized architecture. It includes CI/CD automation and Kubernetes orchestration for efficient deployment. The system also provides REST APIs and queue-based processing for performance.

Scope Includes:

- Docker-based containerization
- CI/CD pipeline implementation
- Kubernetes deployment and orchestration
- Backend API development

Out of Scope:

- Advanced frontend UI design
- Integration with external marketing tools

Proposed Solution

1. Key Features

The proposed system provides a robust and scalable solution using containerization and DevOps practices. Each component of the application is deployed as an independent container, ensuring modularity and ease of maintenance.

The system supports automated deployment through CI/CD pipelines, reducing manual effort and improving reliability. Kubernetes ensures scalability by automatically adjusting resources based on demand.

Key Features:

- Docker-based containerized architecture
- Automated CI/CD pipeline
- Kubernetes-based orchestration
- Campaign management system
- Redis caching and queue processing
- Secure authentication and API handling

2. Overall Architecture / Workflow

The overall architecture of the system follows a containerized DevOps workflow.

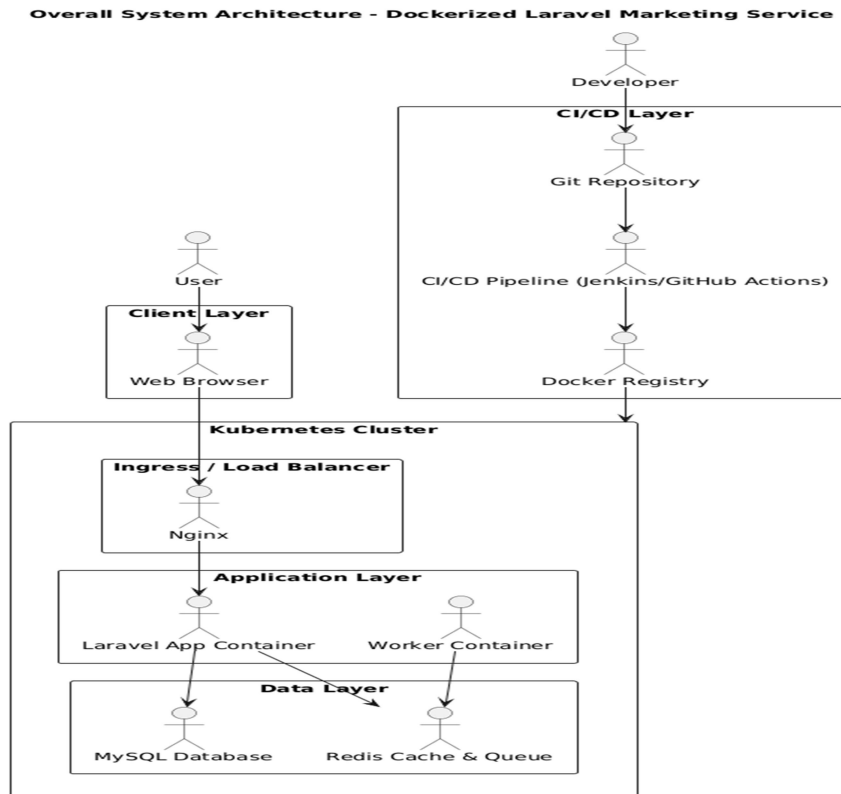


Figure 1: Overall system Architecture Diagram

Overall Architecture Diagram-

the system is divided into multiple layers including the Client Layer, CI/CD Layer, Kubernetes Cluster, Application Layer, and Data Layer. The user interacts with the system through a web browser, which sends requests to the Nginx web server acting as a reverse proxy. The Nginx server forwards these requests to the Laravel Application Container where the core business logic is executed.

Component Diagram - Marketing Laravel Service

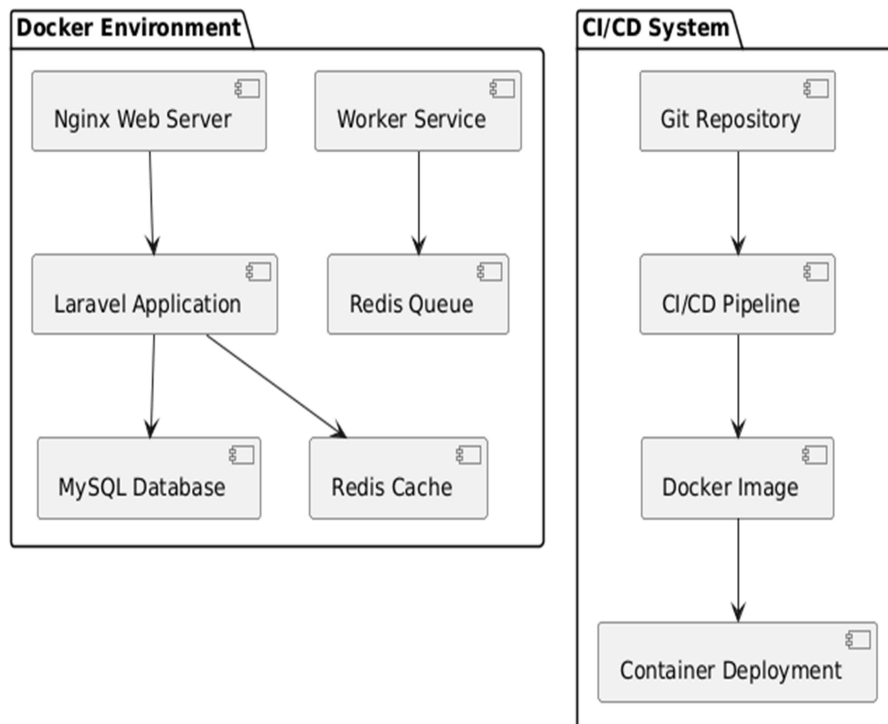


Figure 2: Component Diagram

Component Diagram-

the system is composed of multiple independent components such as the Laravel application, MySQL database, Redis cache, worker services, and Nginx server. These components are deployed as separate Docker containers, ensuring modularity, scalability, and ease of maintenance. The Laravel application communicates with MySQL for persistent data storage and Redis for caching and queue management.

Sequence Diagram - Campaign Execution Workflow

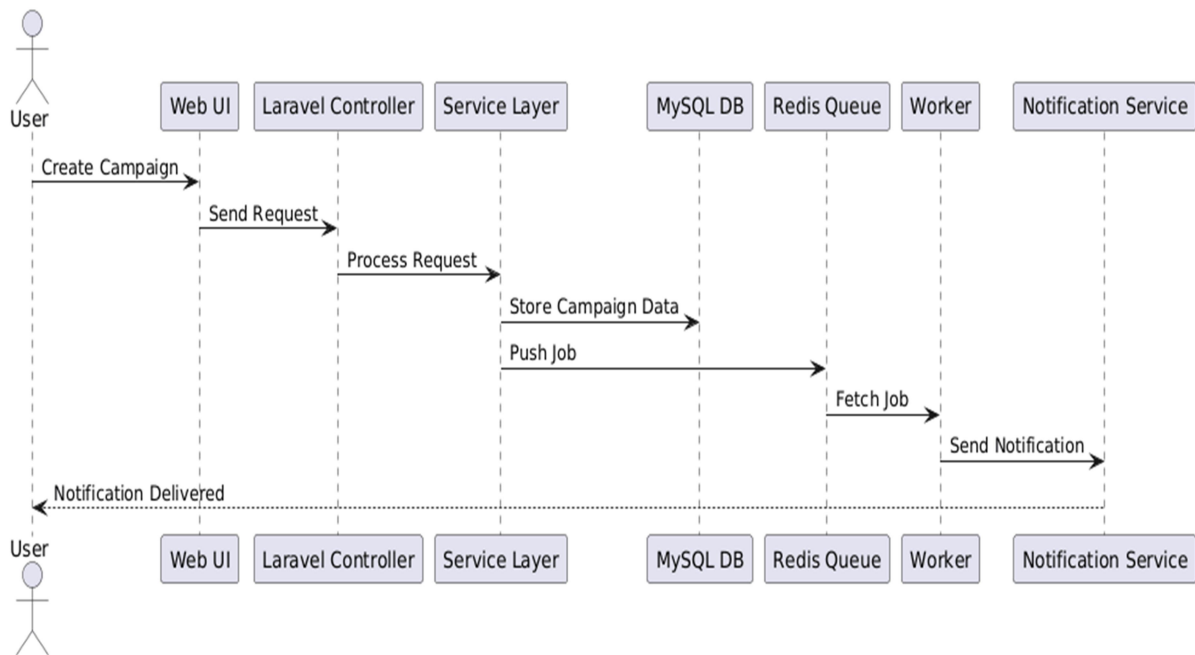


Figure 3: Sequence Diagram

Sequence Diagram-

When a user creates a marketing campaign, the request flows from the user interface to the Laravel controller, then to the service layer, and finally gets stored in the database. Simultaneously, a job is pushed to the Redis queue, which is processed by background workers responsible for sending notifications.

Activity Diagram - Marketing Campaign Flow

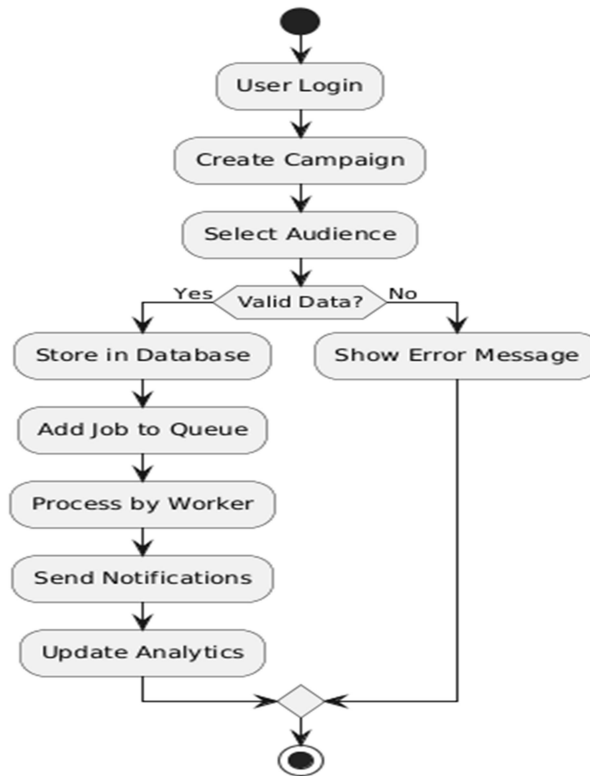


Figure 4: Activity Diagram

Activity Diagram-

The operational workflow is further explained using **Activity Diagram**. It shows the step-by-step flow starting from user login, campaign creation, validation, database storage, queue processing, and notification delivery. Decision points such as data validation ensure system reliability and correctness.

Deployment Diagram - Docker & Kubernetes Setup

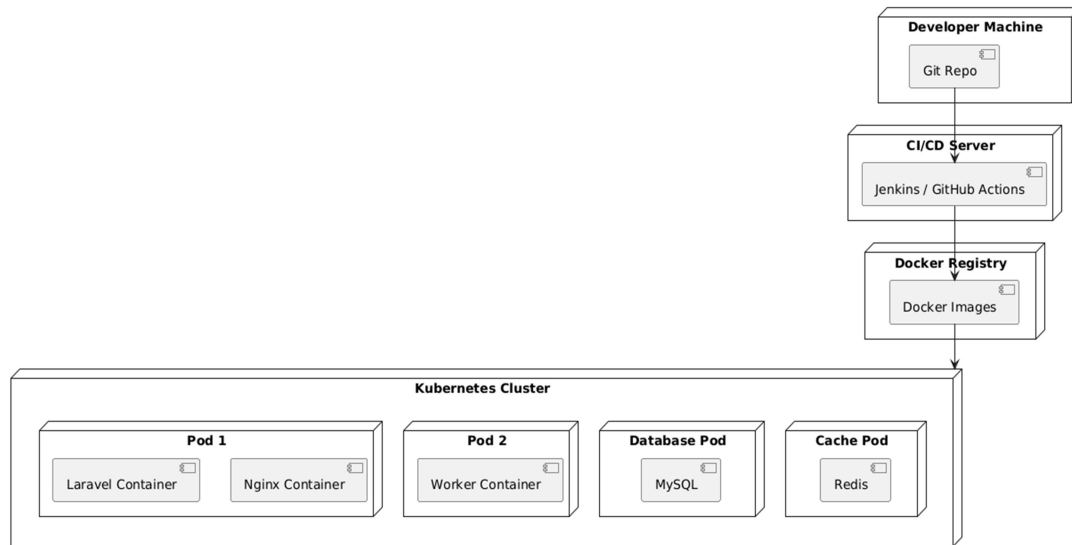


Figure 5: Deployment Diagram

Deployment Diagram-

represents the physical deployment of the system. The application is containerized using Docker and deployed on a Kubernetes cluster. The CI/CD pipeline automatically builds Docker images and pushes them to a registry, from where Kubernetes pulls and deploys them into pods. Kubernetes ensures scalability through auto-scaling and reliability through self-healing mechanisms.

3. Tools & Technologies Used

The project uses modern tools and technologies to ensure efficient development and deployment.

Technologies Used:

- Backend: PHP Laravel
- Containerization: Docker
- CI/CD: Jenkins / GitHub Actions
- Orchestration: Kubernetes
- Database: MySQL
- Caching: Redis
- Web Server: Nginx
- Version Control: Git

These tools together create a robust DevOps ecosystem that improves performance, scalability, and reliability.

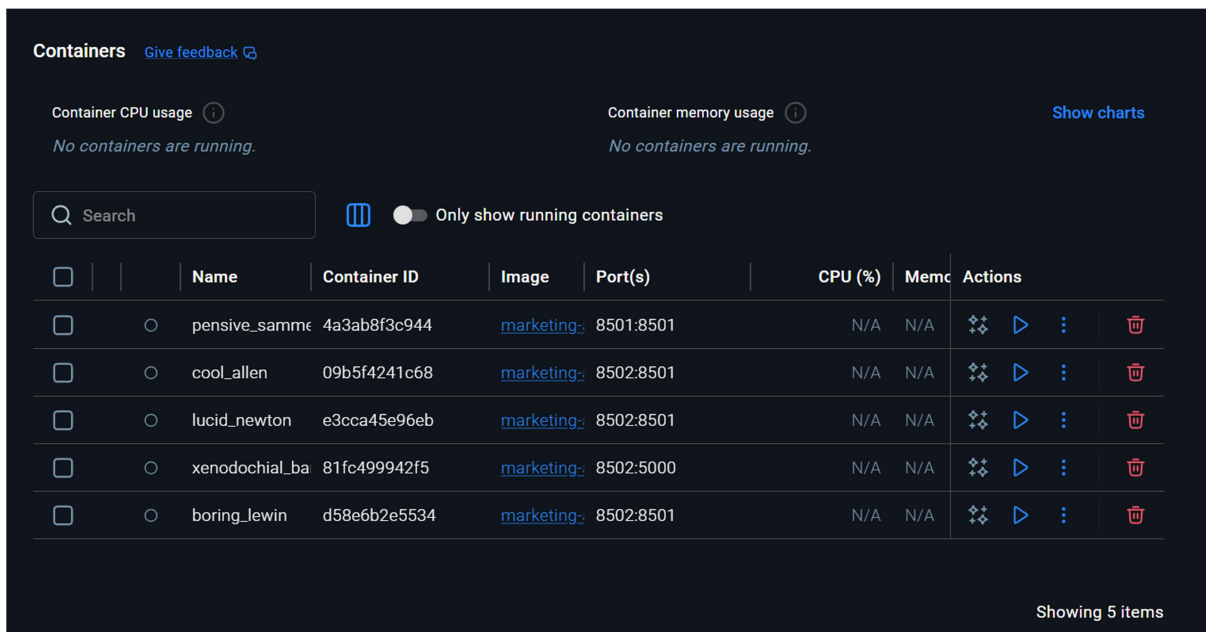
Results & Output

1. Screenshots / Outputs

The project successfully demonstrates the implementation of a Dockerized marketing service. Various outputs can be included to showcase system functionality.

Outputs Include:

- Running Docker containers
- Kubernetes pods and services
- CI/CD pipeline execution
- Application dashboard



The screenshot shows the Docker Desktop interface. At the top, there are sections for 'Container CPU usage' and 'Container memory usage', both indicating 'No containers are running.' Below these are search bars and a toggle for 'Only show running containers'. The main part of the interface is a table of running containers.

	Name	Container ID	Image	Port(s)	CPU (%)	Memc	Actions
☐	pensive_samm	4a3ab8f3c944	marketing-	8501:8501	N/A	N/A	
☐	cool_allen	09b5f4241c68	marketing-	8502:8501	N/A	N/A	
☐	lucid_newton	e3cca45e96eb	marketing-	8502:8501	N/A	N/A	
☐	xenodochial_ba	81fc499942f5	marketing-	8502:5000	N/A	N/A	
☐	boring_lewin	d58e6b2e5534	marketing-	8502:8501	N/A	N/A	

Showing 5 items

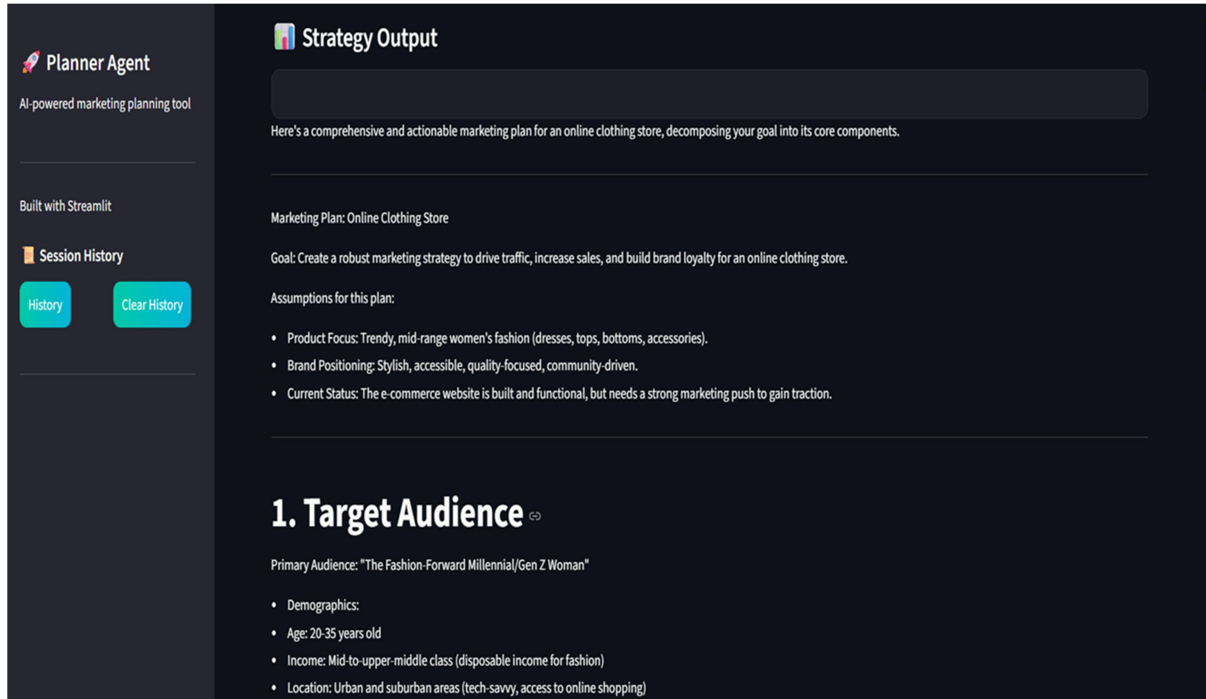
These outputs validate the correct implementation of the system and demonstrate its working.

2. Reports / Dashboards / Models

The system provides dashboards and reports to monitor application performance and campaign analytics. Users can track campaign performance and system health.

Reports Include:

- Campaign analytics dashboard
- CI/CD build and deployment reports
- Kubernetes monitoring metrics



These reports help in analyzing system performance and identifying issues.

3. Key Outcomes

The project achieves several important outcomes that highlight its success.

Key Outcomes:

- Fully automated deployment system
- Improved scalability using Kubernetes
- Reduced manual errors through CI/CD
- Enhanced performance using Redis
- Consistent environment using Docker

Conclusion

The Dockerized Marketing PHP Laravel Service project successfully demonstrates the implementation of modern DevOps practices. By integrating Docker, CI/CD pipelines, and Kubernetes, the system achieves high scalability, reliability, and efficiency.

One of the key learnings from this project is the importance of containerization in ensuring consistency across environments. The use of CI/CD pipelines shows how automation can significantly reduce deployment time and improve system reliability. Kubernetes plays a crucial role in scaling and managing containers efficiently.

Overall, this project provides a strong understanding of DevOps concepts and their practical implementation in real-world applications.

Future Scope & Enhancements

The project can be further enhanced by adding advanced features and improvements.

Future Enhancements:

- AI-based marketing analytics
- Email/SMS integration
- Monitoring using Grafana and Prometheus
- Multi-region deployment
- Advanced security mechanisms

These enhancements will make the system more robust, scalable, and suitable for enterprise-level applications.